

CS Study

Spring - 공통 설정 사항

목차

1

설정 파일 분리

2

Swagger

3

Jacoco

목차

1

설정 파일 분리

2

Swagger

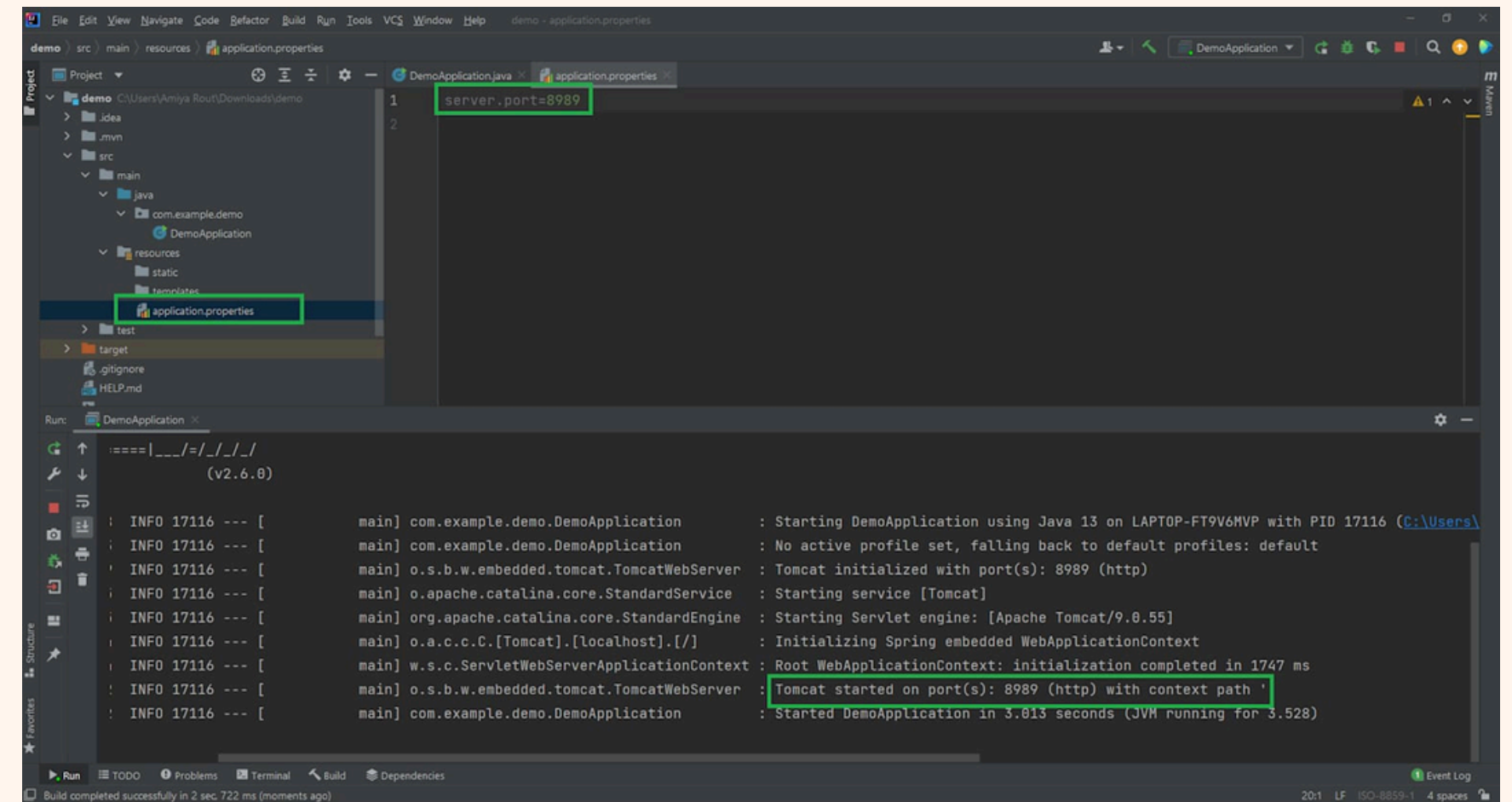
3

Jacoco

설정 파일이란?

설정 파일 개념 및 분리 이유

- 애플리케이션 실행 시 **필요한 환경값들**
외부에 분리해 둔 파일
- 환경 값 수정 시 **재컴파일 및 재배포 방지**
- **환경별 독립성 보장**
- **민감 정보 보호**



설정 파일 환경별 분리

환경별 설정 파일 분리 방법

- **spring.config.import** 사용
 - 기능별 설정 파일 생성
 - application.properties에 import
- **spring.profiles.include** 사용
 - 기능별 설정 파일 생성
 - application.properties에 include

```
# application-db.properties
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=admin
spring.datasource.password=1234
```

```
# application.properties

# application-db.properties 파일을 가져옵니다.
spring.config.import=classpath:application-db.properties

# 다른 설정들도 추가로 import 가능
# spring.config.import=classpath:application-db.properties,
server.port=8080
```


설정 파일 기능별 분리

기능별 설정 파일 분리 방법

- **환경별 설정 파일 분리** 구조 만들기
- **공통 설정 파일**에 공통 값 추가
- **특정 Profile 활성화**

```
src/main/resources/  
├─ application.yml           ← 공통 설정  
├─ application-dev.yml       ← 개발 환경  
├─ application-test.yml      ← 테스트 환경  
└─ application-prod.yml      ← 운영 환경
```

특정 Profile 활성화 방법

특정 Profile 활성화 방법

- **spring.profiles.active** 설정
 - **spring.profiles.active** 사용
- **JVM 옵션 or Command Line** 인자 전달
 - 배포 시 표준 방식
- **IDE 실행 환경** 설정

```
# application.yml
spring:
  profiles:
    active: dev
```

A. JVM 옵션 (**-D**)

시스템 속성(System Property)으로 전달하는 방식이다.

```
java -jar -Dspring.profiles.active=prod my-app.jar
```

B. 커맨드 라인 인자 (**--**)

스프링 부트 인자로 전달하는 방식이다.

```
java -jar my-app.jar --spring.profiles.active=prod
```

설정 파일 기능별 + 환경별 분리

설정 파일 기능별 + 환경별 분리

- **application.yml에서 그룹핑**
 - spring.profiles.group 사용
 - 공통 속성 역시 추가
- **각 환경별 기능별 파일 생성**

```
resources/  
├─ application.yml           (메인: 공통 및 그룹핑 정의)  
├─ application-dev-db.yml    (개발용 DB 설정)  
├─ application-dev-security.yml (개발용 보안 설정)  
├─ application-prod-db.yml    (운영용 DB 설정)  
└─ application-prod-security.yml (운영용 보안 설정)
```


설정 파일 기능별 + 환경별 분리

설정 파일 기능별 + 환경별 분리

- 그룹핑 된 설정들 **동일하게 profile 활성화하면 전부 적용**됨
- application.yml 먼저 읽음
- dev로 **그룹핑된 설정 파일들 모두 읽어 합쳐서 설정 완성**

```
# application.yml
spring:
  profiles:
    group:
      # 'dev' 프로파일 활성화 시 -> dev-db, dev-security 파일 로드
      dev:
        - "dev-db"
        - "dev-security"

      # 'prod' 프로파일 활성화 시 -> prod-db, prod-security 파일 로드
      prod:
        - "prod-db"
        - "prod-security"

# --- [공통 설정] 모든 DB가 공유하는 설정 ---
datasource:
  driver-class-name: com.mysql.cj.jdbc.Driver
  hikari:
    maximum-pool-size: 20
    connection-timeout: 30000
```

설정 파일의 속성 값 사용 방법

설정 파일의 속성 값 사용 방법

- @ConfigurationProperties
 - 설정 파일의 **특정 prefix로 시작**하는 값들
자바 클래스에 통째로 맵핑하는 방식
 - **복잡한 데이터 구조**를 쉽게 매핑 가능
 - **재사용에 용이**

```
# 'myapp.mail' 로 시작하는 설정들을 묶음
myapp:
  mail:
    host: smtp.gmail.com
    port: 587
    enabled: true
    receivers: # 리스트(List)도 쉽게 매핑 가능
      - admin@example.com
      - user@example.com
```

```
@ConfigurationProperties(prefix = "myapp.mail") // 접두어 지정
@Component // Bean으로 등록
@Getter @Setter // Getter, Setter 필요 (Lombok 사용 시)
public class MailProperties {

    private String host;
    private int port;
    private boolean enabled;
    private List<String> receivers; // List 자동 매핑
```

목차

1

설정 파일 분리

2

Swagger

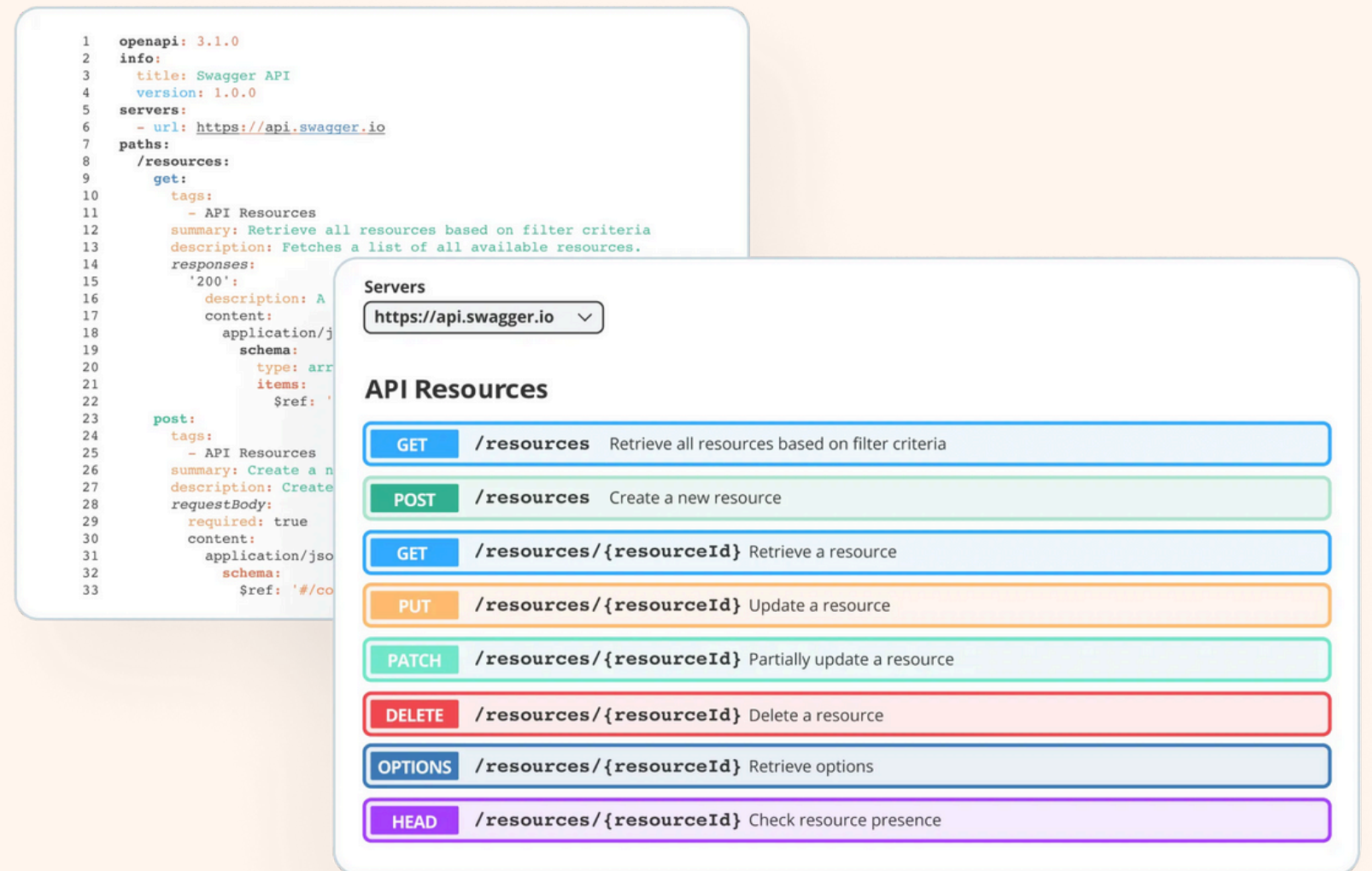
3

Jacoco

Swagger란?

Swagger 개념 및 필요성

- API를 쉽게 설계, 구축, 문서화, 테스트 할 수 있게 도와주는 **API 개발 도구**
- API **문서화 자동화 및 표준화**
- **코드와 문서의 불일치** 문제 해결
- 즉각적인 **API 테스트 지원**



Swagger 적용 방법

의존성 추가

- 프로젝트 설정 파일에 **의존성 추가**
- 의존성 추가만으로 **기본 설정 완료**
- Swagger UI **페이지 접속 가능**

```
dependencies {  
    // ... (기존의 다른 의존성들, e.g., spring-boot-starter-web)  
  
    // Swagger(Springdoc) 의존성 추가  
    implementation 'org.springdoc:springdoc-openapi-starter-webmvc-ui:2.5.0'  
}
```

```
<dependency>  
    <groupId>org.springdoc</groupId>  
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>  
    <version>2.5.0</version>  
</dependency>
```


Swagger 적용 방법

설정 추가

- 설정 추가는 **선택사항**
- 디테일한 커스터마이징
 - UI 접속 경로 변경
 - API 정렬 순서
 - scan 패키지 지정

```
springdoc:  
  swagger-ui:  
    path: /api-docs           # 접속 경로 변경  
    operations-sorter: alpha  # API 정렬  
    tags-sorter: alpha       # 태그 정렬  
    enabled: true            # 활성화 여부  
  api-docs:  
    path: /api-docs-json      # JSON 명세 경로 변경  
    enabled: true  
  packages-to-scan: com.example.myproject.controller # 스캔 패키지
```

Swagger 적용 방법

config 파일 생성

- Swagger UI 페이지의 **간판 만드는 작업**
- 문서의 **제목, 설명, 버전 등** 내용 정의
- **JWT 로그인 시** 추가 코드 필요

```
@Configuration // 1. 스프링 설정 클래스임을 명시
public class SwaggerConfig {

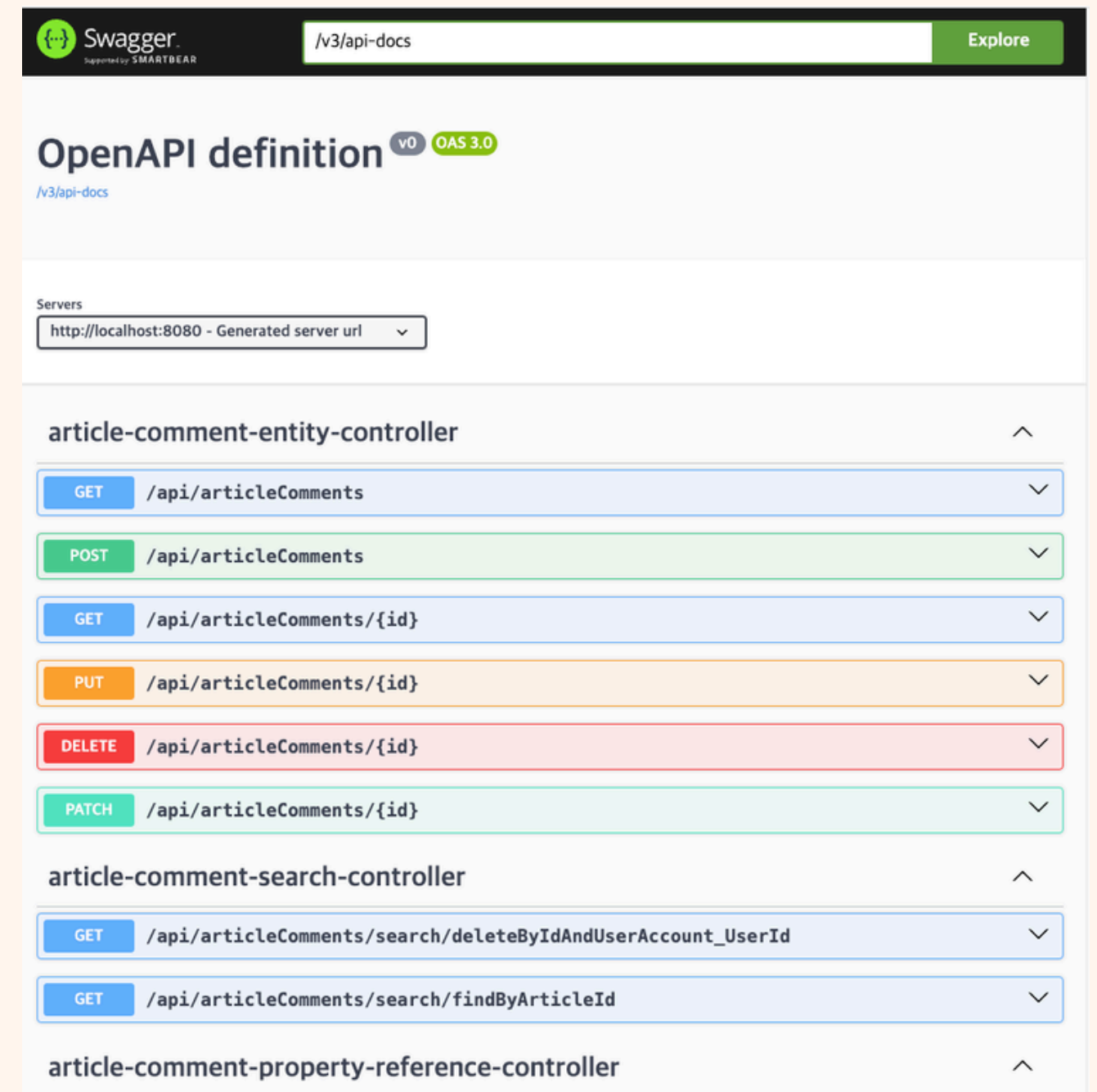
    @Bean
    public OpenAPI openAPI() {
        return new OpenAPI()
            .components(new Components())
            .info(apiInfo());
    }

    private Info apiInfo() {
        return new Info()
            .title("API Test") // 2. API 제목
            .description("Let's practice Swagger UI") // 3. API 설명
            .version("1.0.0"); // 4. API 버전
    }
}
```

Swagger 접속 및 확인

Swagger 접속 및 확인

- <http://localhost:8080/swagger-ui.html>
→ API 문서 확인 가능
- Swagger UI 통해 각 API 엔드포인트 테스트 가능



Swagger 주요 annotation

Swagger 주요 annotation

- **@Tag (API 그룹화)**

- 관련된 API들을 하나의 그룹으로 묶음
- 제목, 설명 붙여줌

- **@Operation (API 설명)**

- 개별 API가 뭐하는 것인지 설명
- 없다면 URL만 나옴

- **@ApiResponse (응답 설명)**

- 서버 반환 HTTP 응답 (JSON) 정의
- 없다면 200 OK만 보여줌

- **@Schema (DTO 설명)**

- DTO 상세히 설명
- DTO 클래스에 작성

목차

1

설정 파일 분리

2

Swagger

3

Jacoco

Jacoco란?

Jacoco 개념 및 특징

- 테스트 코드가 **실제 프로덕션 코드 얼마나 커버했는지** 측정하는 라이브러리
- 코드 얼마나 꼼꼼하게 검증하고 있는지 **수치와 시각적인 리포트로 보여줌**
- 바이트코드 조작
- 상세한 커버리지 지표 제공
- 가독성 높은 리포트

Jacoco 적용 방법

Jacoco 플러그인 설정

- 일반적인 라이브러리와 달리, **빌드 도구의 플러그인 형태로 프로젝트에 주입**
- **build.gradle의 plugins 블록**에 추가
 - 도구 버전 명시 (선택 사항)
- maven은 **pom.xml의 <build> → <plugins> 태그**에 추가

Jacoco 적용 방법

Jacoco Gradle 주요 Task 설정

- Jacoco의 **3가지 Task**는 아래와 같다.
 - **test** : Jacoco task 순서 설정
 - **jacocoTestReport**: 리포트 생성
 - **jacocoTestCoverageVerification**: 커버리지 기준 검증
- "**테스트 실행→결과 리포트 생성→기준 통과 여부 검사**"라는 전체 파이프라인 구성

Jacoco 적용 방법

Jacoco Gradle 주요 Task 설정

- **test : Jacoco task 순서 설정**
 - JUnit 테스트 실행
 - 이때 Jacoco가 코드 실행 정보 수집
 - [테스트 실행 → 리포트 생성 → 검증]

순서 보장되도록 명시적인 설정

```
// 1. test Task가 끝난 뒤에 리포트 생성이 실행되도록 설정
test {
    finalizedBy jacocoTestReport
}

// 2. 리포트 생성이 끝난 뒤에 검증이 실행되도록 설정
jacocoTestReport {
    finalizedBy jacocoTestCoverageVerification
}
```

Jacoco 적용 방법

Jacoco Gradle 주요 Task 설정

- **jacocoTestReport: 리포트 생성**
 - 테스트 실행 후 생성된 바이너리
커버리지 데이터 기반으로 리포트
를 생성하는 Task

```
jacocoTestReport {  
    reports {  
        html.required = true // 사람이 보기 위해 필수  
        xml.required = true  // CI/CD, SonarQube 연동 시 필수  
        csv.required = false // 보통 false로 설정  
    }  
  
    // 리포트 생성 후 바로 커버리지 검증 단계로 넘어가도록 설정 (  
    // finalizedBy 'jacocoTestCoverageVerification'  
}
```

Jacoco 적용 방법

Jacoco Gradle 주요 Task 설정

- **jacocoTestCoverageVerification:**

커버리지 기준 검증

- 프로젝트에서 정한 최소한의 **커버리지**
기준을 만족하는지 검사
- 기준에 미달하면 **빌드 자체를 실패시켜**
배포를 막음

```
jacocoTestCoverageVerification {  
    violationRules {  
        rule {  
            enabled = true  
            element = 'CLASS' // 클래스 단위로 체크  
  
            // 조건 1: 브랜치 커버리지 90% 이상  
            limit {  
                counter = 'BRANCH'  
                value = 'COVEREDRATIO'  
                minimum = 0.90  
            }  
  
            // 조건 2: 라인 커버리지 80% 이상  
            limit {  
                counter = 'LINE'  
                value = 'COVEREDRATIO'  
                minimum = 0.80  
            }  
        }  
    }  
}
```


Jacoco 적용 방법

Jacoco Gradle 경로 제외

- **비즈니스 로직과 무관한 코드** 포함되면?
- 전체 커버리지 점수 깎여 **왜곡된 지표** 나옴
- 제외 파일
 - DTO / Entity
 - Config 파일
 - Exception 등

```
jacocoTestReport {
    dependsOn test

    // 리포트 생성 시 제외할 파일 패턴 설정
    afterEvaluate {
        classDirectories.setFrom(files(classDirectories.files.collect {
            fileTree(dir: it, exclude: [
                "**/Q*",           // 1. QueryDSL
                "**/*Application*", // 2. 메인 애플
                "**/*Config*",      // 3. 설정 파일
                "**/*Dto*",         // 4. DTO (클래
                "**/*Request*",     // 5. Request I
                "**/*Response*",    // 6. Response
                "**/*Exception*",   // 7. 예외 클래
                "**/entity/**"      // 8. 특정 패키
            ])
        }))
    }
}

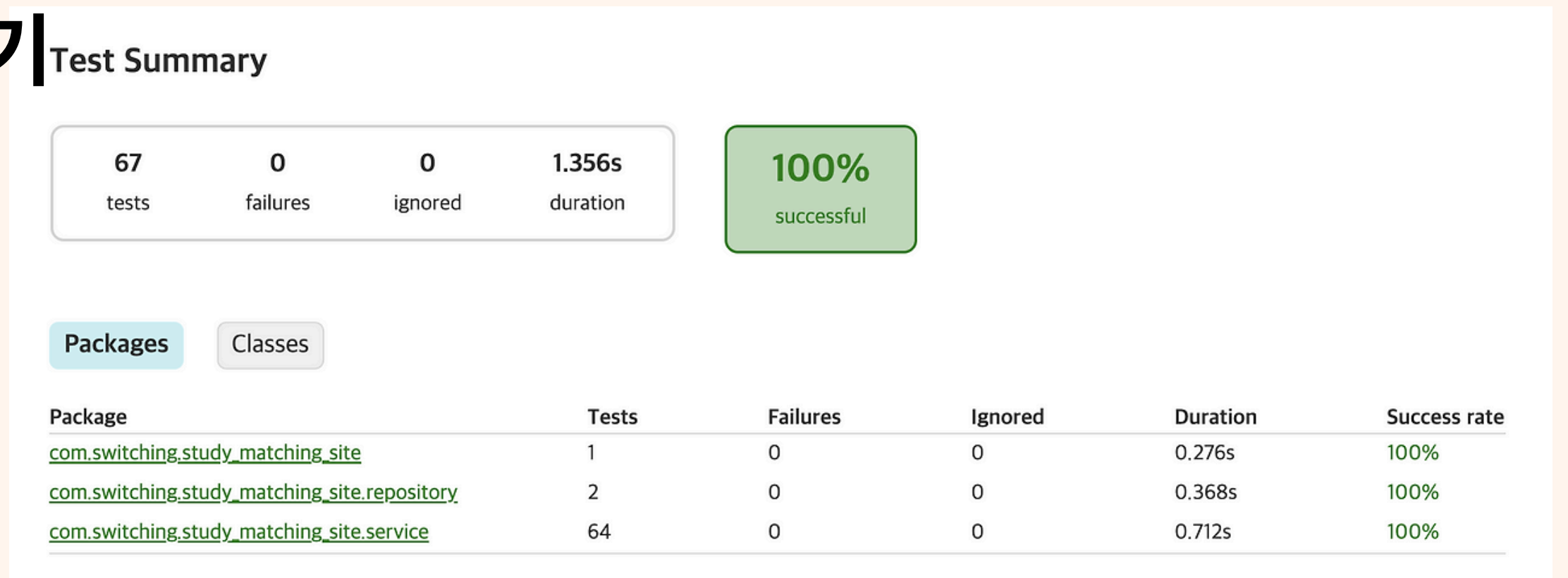
jacocoTestCoverageVerification {
    // 검증 시 제외할 파일 패턴 설정 (위와 동일한 패턴 적용)
    violationRules {
        rule {
            element = 'CLASS'

            // excludes 속성을 사용하여 제외 패턴 등록
            excludes = [
```

Jacoco 리포트 보기

Jacoco Gradle 리포트 보기

- 메인 화면
- 전체 테스트 개수, 걸린 시간, 테스트 실패율 등 표시



Jacoco 리포트 보기

Jacoco Gradle 리포트 보기

- 각 클래스별 리포트
- 각 커버리지 항목당 총 개수, 놓친 개수 표시

 [jacoco-on-gradle-sample](#) >  [com.woowahan.thdeng.test.jacoco](#) >  KotlinFoo

KotlinFoo

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
 hello(String)		62%		33%	3	4	2	6	0	1
 callMe()		0%		n/a	1	1	2	2	1	1
 KotlinFoo()		100%		n/a	0	1	0	1	0	1
Total	17 of 35	51%	4 of 6	33%	4	6	4	9	1	3

Jacoco 리포트 보기

Jacoco Gradle 리포트 보기

- 코드 파일
- 커버된 라인은 초록색
- 놓친 라인은 빨간 색
- 일부만 테스트된 것은 노란색

 [jacoco-on-gradle-sample](#) >  [com.woowahan.thdeng.test.jacoco](#) >  Kotlin

KotlinFoo.kt

```
1. package com.woowahan.thdeng.test.jacoco
2.
3. class KotlinFoo {
4.
5.     fun hello(name: String): String = when {
6.         name == "Hello" -> "world"
7.         name == "펍" -> "하"
8.         name.length > 5 -> "TOO LONG"
9.         else -> "no one"
10.    }
11.
12.    fun callMe() {
13.        println("Please, call me")
14.    }
15.
16. }
```